

Stochastic Gradient Descent

A STOCHASTIC APPROXIMATION METHOD¹

BY HERBERT ROBBINS AND SUTTON MONRO

University of North Carolina

1. Summary. Let $M(x)$ denote the expected value at level x of the response to a certain experiment. $M(x)$ is assumed to be a monotone function of x but is unknown to the experimenter, and it is desired to find the solution $x = \theta$ of the equation $M(x) = \alpha$, where α is a given constant. We give a method for making successive experiments at levels $x_1, x_2, \dots$ in such a way that x_n will tend to θ in probability.

2. Introduction. Let $M(x)$ be a given function and α a given constant such that the equation

$$(1) \quad M(x) = \alpha$$

has a unique root $x = \theta$. There are many methods for determining the value of θ by successive approximation. With any such method we begin by choosing one or more values $x_1, \dots, x_r$ more or less arbitrarily, and then successively obtain new values x_n as certain functions of the previously obtained $x_1, \dots, x_{n-1}$, the values $M(x_1), \dots, M(x_{n-1})$, and possibly those of the derivatives $M'(x_1), \dots, M'(x_{n-1})$, etc. If

$$(2) \quad \lim_{n \rightarrow \infty} x_n = \theta,$$

1951

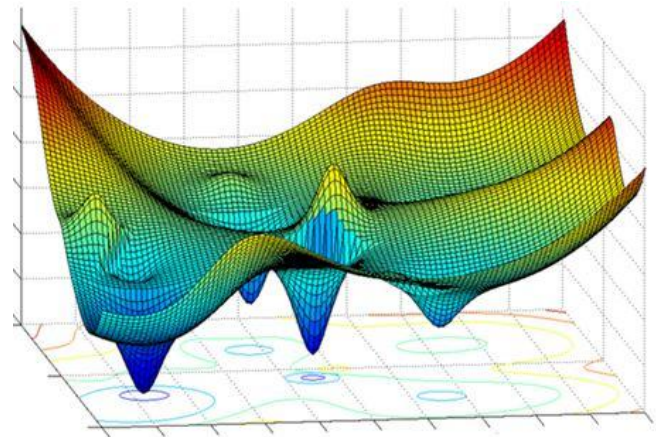
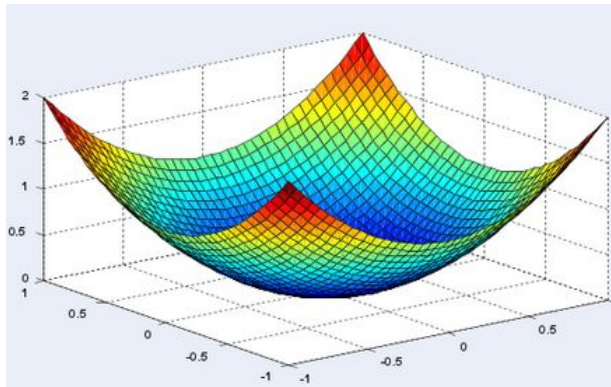
Stochastic Gradient descent

- Powers nearly **all deep learning systems** including image recognition, language models, and recommendation engines including large-scale neural networks (and LLMs)
- Enables **online learning** where models update with streaming data
- Successfully **scales** to massive datasets too large to fit in memory
- Provides **natural regularization** through noise in gradient estimates
- Forms the foundation for modern **optimizers** like Adam, RMSprop and AdaGrad
- Critical enabler for **distributed and parallel training** across multiple devices
- Underpins breakthroughs in NLP, computer vision, and reinforcement learning

Gradient descent

The gradient descent method can minimize **any function** that is

- A. real
- B. continuous
- C. convex with a unique minimum



Gradient descent

To find the local minima of a function you find the parameter for which the derivative is zero. With Gradient descent we find the function minima **iteratively**

Let $f(w)$ be a continuous real function that we want to minimize.

If $f(w)$ has a derivative and is convex, for α small
We calculate iteratively

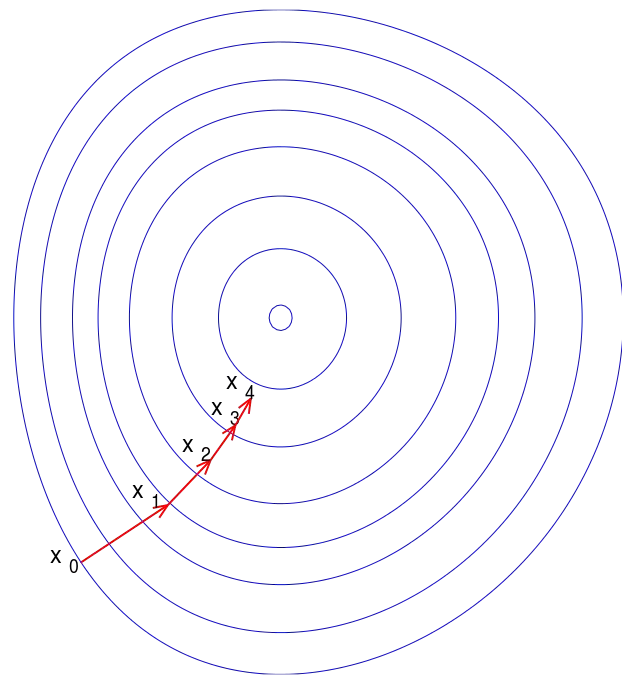
$$w(t+1) = w(t) - \alpha \nabla f(w(t))$$

Gradient in $w(t)$

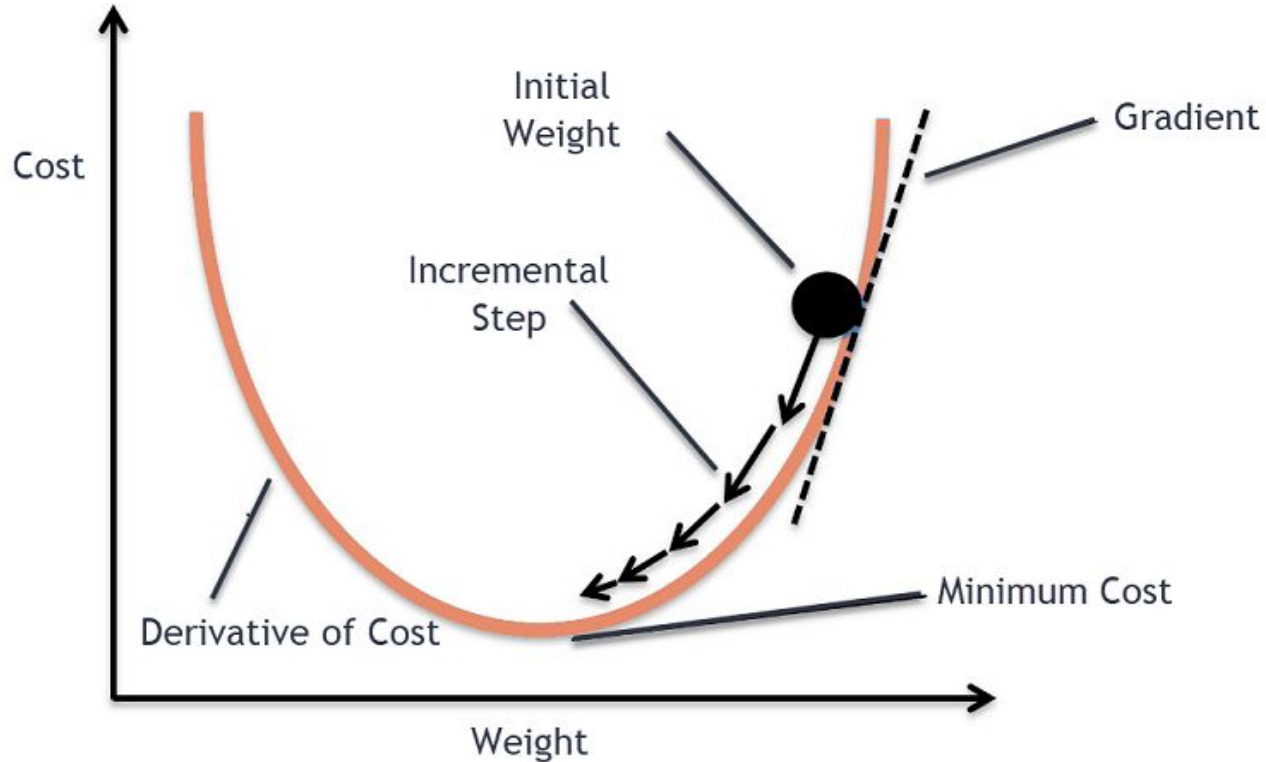
It can be proven that $w(t)$ converges towards the minima of f

And we have for each iteration:

$$f(w(0)) \geq f(w(1)) \geq f(w(2)) \geq \dots,$$



Gradient descent



Gradient descent

Demo

- Take a polynomial function with degree n ($n = 3, 4, \dots$)
 - $f(x) = \sum a_i x^i$
- plot the function on an interval that has at least one minimum
- Calculate the exact value of the minimum using the derivative.

Programmatically

- Define x_0, α
- Calculate $x_{t+1} = x_t - \alpha * d fct(x_t) / dx$
- Plot x_t
- and plot $\Delta_t = x_{t+1} - x_t$

- `np.arange(a, b, step)`
-

see the [colab](#)

Gradient descent for supervised machine learning

In supervised machine learning the function to minimize is the loss function

By minimizing the loss function, we find the optimal parameters of the model that minimize prediction error.

However, we can't just derive the loss function and solve for zero because:

- Most ML loss functions are non-convex with many parameters, making analytical solutions impossible
- For neural networks, the loss landscape is extremely complex with many local minima
- Even for simpler models, closed-form solutions don't exist when dealing with millions of model parameters (Deep NN)
- Matrix operations required for analytical solutions become computationally intractable at scale
- And many loss functions used in practice don't have closed-form solutions

So we need to estimate the gradient : the value of the derivative of the loss function at a certain point

3 types of gradient

There are several ways to calculate or approximate the gradient for each iteration.

Each corresponds to a specific type of gradient descent with impact on convergence, resources and applications

- **Batch:** takes the **whole dataset** to calculate the gradient term. shuffle on each iteration
 - fast and guaranteed convergence
 - memory intense for large datasets
- **Mini Batch:** takes a subset of size N of the whole dataset
 - slower convergence
 - less memory needed
- **Stochastic:** the gradient is estimated on just one random sample of the dataset
 - convergence can be chaotic and slow
 - small memory imprint (IoT, embedded, sensors)
 - online training: the model can be trained on new data as it becomes available

Batch Gradient Descent

Vanilla gradient descent, aka batch gradient descent, computes the gradient of the cost function w.r.t. to the parameters for the entire training dataset:

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta).$$

As we need to calculate the gradients for the whole dataset to perform just one update, batch gradient descent can be very slow and is intractable for datasets that don't fit in memory. Batch gradient descent also doesn't allow us to update our model *online*, i.e. with new examples on-the-fly.

[An overview of gradient descent optimization algorithms](#)

Stochastic Gradient Descent

Stochastic gradient descent (SGD) in contrast performs a parameter update for *each* training example

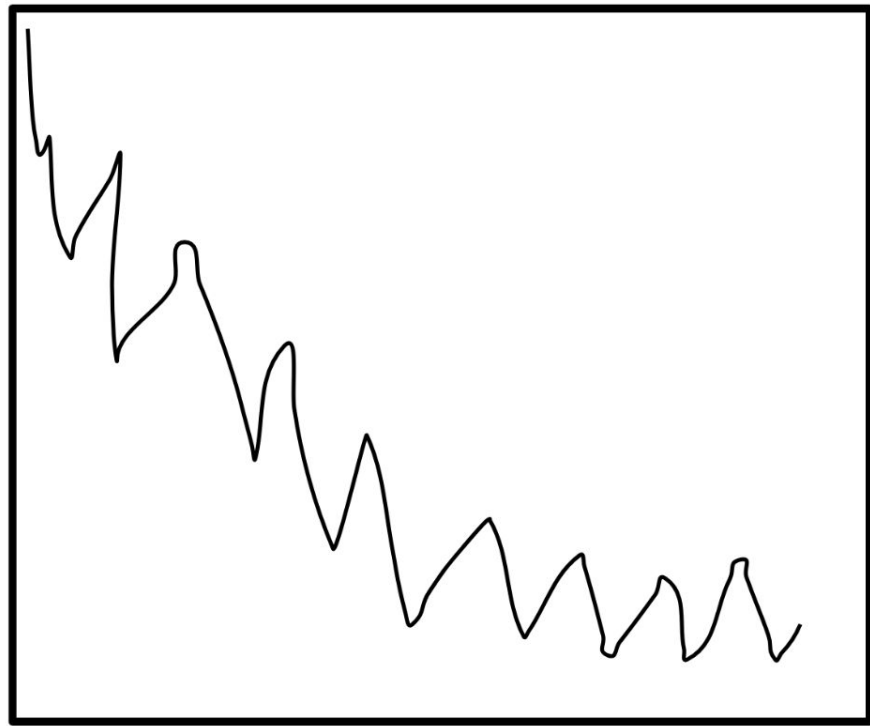
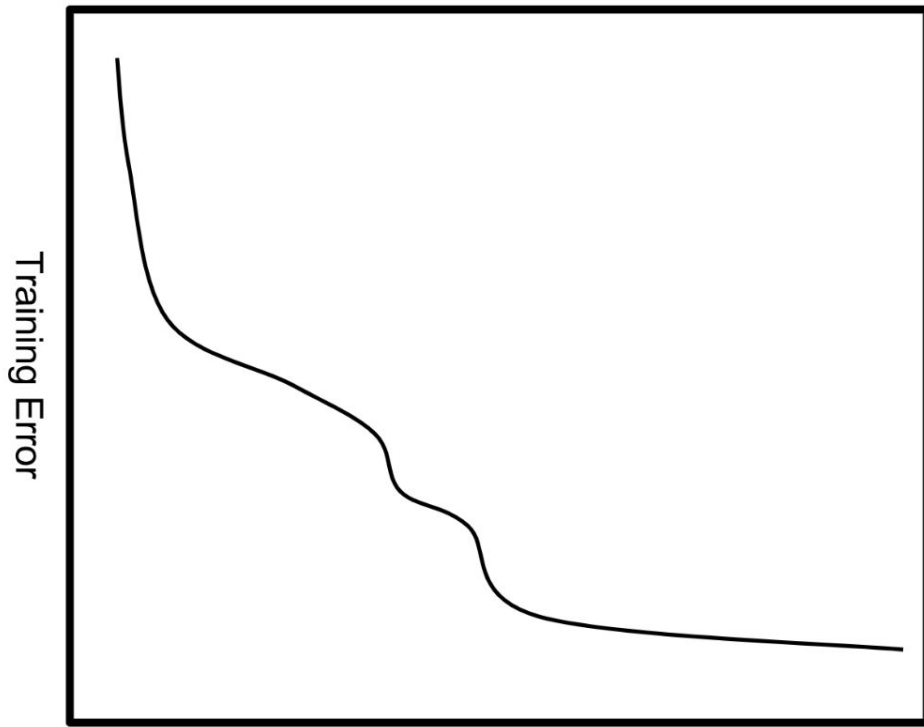
$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i)}; y^{(i)}).$$

- Batch gradient descent performs redundant computations for large datasets, as it recomputes gradients for similar examples before each parameter update.
- SGD does away with this redundancy by performing one update at a time. It is therefore usually much faster and can also be used to learn online.
- SGD performs frequent updates with a high variance that cause the objective function to fluctuate heavily overshooting around the local minimum.
- It has been shown that when we slowly decrease the learning rate, SGD shows the same convergence behaviour as batch gradient descent, almost certainly converging to a local or the global minimum for non-convex and convex optimization respectively.

[An overview of gradient descent optimization algorithms](#)

Batch Gradient Descent

Vanilla gradient descent, aka batch gradient descent, computes the gradient of the cost function w.r.t. to the parameters for the entire training dataset:



Mini batch gradient descent - aka out-of-core

Mini-batch gradient descent finally takes the best of both worlds and performs an update for every mini-batch of n training examples:

$$\theta = \theta - \eta \cdot \nabla_{\theta} J(\theta; x^{(i:i+n)}; y^{(i:i+n)}).$$

- It reduces the variance of the parameter updates, which leads to more stable convergence;
- Makes use of optimized matrix optimizations common to state-of-the-art deep learning libraries that make computing the gradient w.r.t. a mini-batch very efficient.
- Common mini-batch sizes range between 50 and 256, but can vary for different applications.
- Mini-batch gradient descent is typically the algorithm of choice when training a neural network and the term SGD usually is employed also when mini-batches are used.

[An overview of gradient descent optimization algorithms](#)

Stochastic gradient - why does it work

The expectation of the prediction error is the gradient when the samples are i.i.d : independently and identically distributed

The stochastic gradient $\nabla L(y_i, f(x_i))$ is an unbiased estimator of the true gradient.

$$\mathbb{E}_i [\nabla L(y_i, f(x_i))] = \frac{1}{n} \sum_{i=1}^n \nabla L(y_i, f(x_i)) = \nabla E(w)$$

expectation of the
prediction error for
a sample

average of the
prediction error
over all samples

true gradient

Stochastic gradient - MSE loss

Loss function

$$L(y_i, f(x_i)) = (y_i - f(x_i))^2$$

Gradient

$$\nabla_w L = -2(x_i)(y_i - f(x_i))$$

**need to
normalize**

prediction error

Stochastic gradient with regularization

$$E(w, b) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \alpha R(w)$$

$$w \leftarrow w - \eta \left(\alpha \frac{\partial R(w)}{\partial w} + \frac{\partial L(w^T x_i + b, y_i)}{\partial w} \right)$$

η learning rate

α regularization parameter

Scikit - SGDRegressor

`sklearn.linear_model.SGDRegressor`

```
class sklearn.linear_model.SGDRegressor(loss='squared_loss', *, penalty='l2', alpha=0.0001, l1_ratio=0.15,
fit_intercept=True, max_iter=1000, tol=0.001, shuffle=True, verbose=0, epsilon=0.1, random_state=None,
learning_rate='invscaling', eta0=0.01, power_t=0.25, early_stopping=False, validation_fraction=0.1, n_iter_no_change=5,
warm_start=False, average=False)
```

[\[source\]](#)

- **loss**: loss function.
 - 'squared_error' refers to the ordinary least squares fit.
- **penalty**: regularization:
 - L1, L2, elasticnet
- **alpha**: regularization term
- **learning rate**: different mode of updating the learning rate.
- **early stopping**: limit the number of iterations

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDRegressor.html

Scikit - SGDRegressor - learning rate

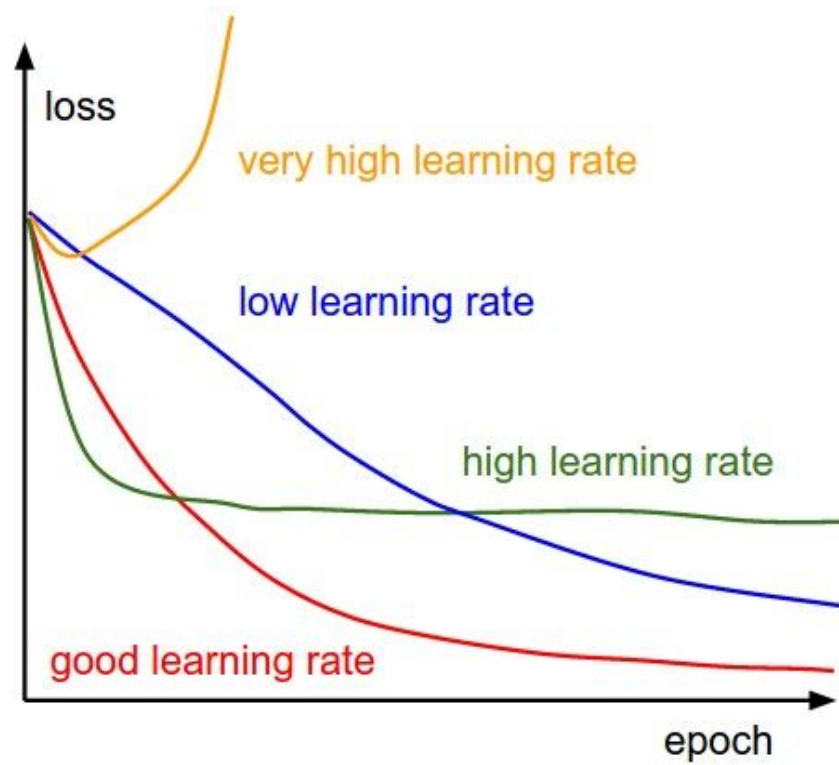
learning_rate: default='invscaling'

- *constant*: $\eta = \eta_0$
- *optimal*: $\eta = 1.0 / (\alpha * (t + t_0))$
- *invscaling*: $\eta = \eta_0 / \text{pow}(t, \text{power_t})$
- *adaptive*: $\eta = \eta_0$, as long as the training keeps decreasing.

eta0, default=0.01

The initial learning rate for the 'constant', 'invscaling' or 'adaptive' schedules. The default value is 0.01. Values must be in the range [0.0, inf).

SGD Regressor



Minibatch

```
from sklearn.linear_model import SGDRegressor
```

```
def iter_minibatches(chunksize):  
    # Provide chunks one by one  
    chunkstartmarker = 0  
    while chunkstartmarker < numtrainingpoints:  
        chunkrows = range(chunkstartmarker, chunkstartmarker+chunksize)  
        X_chunk, y_chunk = .... slice X,y ....  
        yield X_chunk, y_chunk  
        chunkstartmarker += chunksize
```

```
def main():  
    batcherator = iter_minibatches(chunksize=1000)  
    model = SGDRegressor()  
  
    # Train model  
    for X_chunk, y_chunk in batcherator:  
        model.partial_fit(X_chunk, y_chunk)  
  
    # Now make predictions with trained model  
    y_predicted = model.predict(X_test)
```

1. Break down your data (X,y)
into chunks (X_k, y_x)

2. Call `partial_fit` k times

Demo Cars93 - SGD Regressor - GridSearchCV

<https://colab.research.google.com/drive/1EDjgLYCtcudxcSfhSNbBjopig1GGeHHk?usp=sharing>

GridSearchCV

Influence of learning rate on convergence and minimum

Influence of batch size on mini-batch using `partial_fit`

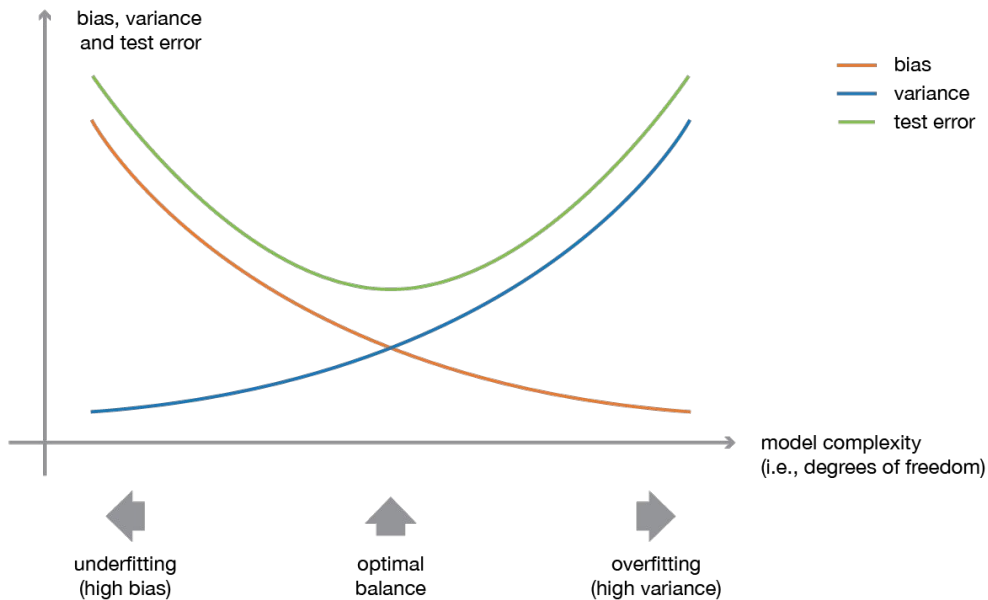
early stopping

influence of random seed

Bias - Variance trade off - Overfitting

We want a model that performs well on unseen data

- low bias : good predictions
- low variance : not too sensitive to training data



Hands on work on SGD - california housing

GridSearchCV

Influence of learning rate on convergence and minimum

Influence of batch size on mini-batch using `partial_fit`

early stopping

influence of random seed

Lectures et références

- [Efficient BackProp Yann LeCun](#)
- [Vue d'ensemble de la méthode du gradient stochastique](#)
- [Tips on practical use](#) (scikit)
- [A Gentle Introduction to Mini-Batch Gradient Descent and How to Configure Batch Size](#) By Jason Brownlee
- [An overview of gradient descent optimization algorithms](#) By Sebastian Ruder
- [Early stopping of Stochastic Gradient Descent — scikit-learn 0.23.2 documentation](#)